

ProfilePeek — IDOR via Header Scope

☆☆☆☆ fácil

1 punto

⌚ 00h 17m 59s

CWE-639 · IDOR

01 · CONTEXTO

ProfilePeek v0.1 es una aplicación web de directorio interno de empleados desplegada en un contenedor Docker. Expone un sistema de perfiles identificados numéricamente (`/profile?uid=N`) donde la API interna utiliza *headers de identidad* para gestionar la sesión activa. El objetivo del reto es acceder a un campo restringido de un perfil privilegiado sin tener las credenciales correspondientes.

Plataforma: **WHOAMI-LABS.COM** · IP objetivo: **172.17.0.2:5000** ·
Fecha: **2026-09-25**

02 · RECONOCIMIENTO

Descubrimiento de host y servicios

Partiendo de la subred Docker (`172.17.0.0/16`), se realiza un ping sweep para localizar hosts activos, seguido de un escaneo de puertos completo con detección de versión.

```
kali@kali - reconocimiento

└─(kali@kali)-[~]
└─$ nmap -v -sn 172.17.0.0/16
Host: 172.17.0.2 () Status: Up

└─(kali@kali)-[~]
```

```
└─$ nmap -v -sS -sV -p- 172.17.0.2
PORT      STATE SERVICE VERSION
5000/tcp  open  http   Werkzeug/Flask (Python)
```

Resultado clave: un único servicio HTTP en el puerto **5000**, corriendo sobre Flask/Werkzeug. No hay HTTPS, el tráfico viaja en claro.

03 · ENUMERACIÓN

Reconocimiento de la aplicación web

La página `/about` revela la versión de la aplicación y la lista completa de endpoints sin ningún mecanismo de autenticación:

```
# Respuesta de /about
ProfilePeek v0.1 – demo de perfiles.
API interna usa headers de identidad.

Endpoints: /, /login, /profiles, /profile?uid=N, /about
```

Dos señales de alerta inmediatas: la arquitectura interna queda expuesta públicamente, y se confirma que la API gestiona identidad mediante *headers* — un vector clásico de IDOR si no se valida correctamente en el servidor.

Accediendo a `/profiles` se obtiene el directorio completo de 20 empleados sin autenticación adicional. La entrada más relevante:

uid	user	nombre
...	...	...
17	carol	Carol CISO Admin ← objetivo principal

El cargo "CISO Admin" señala una cuenta privilegiada con acceso a campos restringidos. Se convierte en el objetivo de la fase de explotación.

Login con credenciales demo

La propia página de login expone en claro las credenciales de demostración: `alice / alice1`. Se inicia sesión como `alice (uid=1)`. La sesión activa queda marcada en la aplicación como `uid sesión=1`.

04 · EXPLOTACIÓN

IDOR — Insecure Direct Object Reference

CWE-639 · Authorization Bypass Through User-Controlled Key

El endpoint `/profile?uid=N` utiliza un identificador numérico directamente controlable por el usuario sin verificar si la sesión autenticada tiene autorización para acceder al recurso solicitado.

El vector de ataque es trivial: siendo `alice (uid=1)` la sesión activa, basta con modificar el parámetro `uid` en la URL para apuntar al perfil de la CISO (`uid=17`). La aplicación devuelve el perfil completo incluyendo el campo restringido, sin validar que la sesión corresponda al propietario del recurso ni a un rol privilegiado.

01 Login en `/login` con `alice / alice1` — credenciales expuestas en claro en la propia página de login.

02 Enumeración del directorio en `/profiles` → identificación de `uid=17` como Carol CISO Admin.

03 Manipulación manual del parámetro: navegar a `/profile?uid=17` mientras la sesión activa es `uid sesión=1` (`alice`).

04 La aplicación devuelve el perfil completo de carol incluyendo el campo restringido con la flag.

GET http://172.17.0.2:5000/profile?uid=17

← respuesta del servidor:

Perfil #17

Usuario: carol

Nombre: Carol CISO Admin

Email: carol.ciso@corp.local

Campo restringido: **BURP{idor_header_scope}**

Sesión actual: alice (uid sesión=1) ← alice accede a datos de carol

La línea "*Sesión actual: alice (uid sesión=1)*" confirma de forma inequívoca que la aplicación **no verifica** que la sesión activa coincida con el uid del perfil solicitado. Cualquier usuario puede leer cualquier perfil, incluidos campos marcados como restringidos.

05 · FLAG



PROFILEPEEK · CAMPO RESTRINGIDO UID=17

BURP{idor_header_scope}

06 · IMPACTO

CONFIDENCIALIDAD

Alta — cualquier usuario autenticado puede leer datos de todos los empleados, incluidos cargos directivos.

DATOS EXPUESTOS

Nombre completo, email corporativo, cargo interno y campos restringidos de todos los perfiles.

PRIVILEGIO NECESARIO

Bajo — cualquier cuenta activa (incluso de demostración) es suficiente para explotar la vulnerabilidad.

COMPLEJIDAD DE ATAQUE

Mínima — no se requieren herramientas especiales; basta con modificar la URL en el navegador.

En un entorno real, un atacante con acceso a cualquier cuenta de bajo privilegio podría exfiltrar la totalidad del directorio de empleados y obtener información sensible de roles de administración, seguridad y finanzas.

07 · REMEDIACIÓN

Medidas correctoras

Prioridad	Medida	Detalle técnico
ALTA	Validación de autorización en el servidor	Verificar en cada request que <code>session['uid']</code> coincide con el uid solicitado, o que el usuario tiene un rol privilegiado. Nunca confiar en el parámetro client-side.
ALTA	Eliminar credenciales demo de la UI	Las credenciales <code>alice/alice1</code> expuestas en la página de login eliminan cualquier barrera de entrada. Eliminarlas o moverlas a documentación privada.
ALTA	Eliminar divulgación de endpoints	La página <code>/about</code> no debería revelar la arquitectura de endpoints ni los mecanismos de autenticación internos.
MEDIA	Control de acceso basado en roles (RBAC)	Los campos marcados como "restringidos" deben requerir un rol explícito (p. ej. <code>CISO</code> o <code>admin</code>) en lugar de ocultarse solo con lógica frontend.
MEDIA		

Prioridad	Medida	Detalle técnico
	Restringir acceso a /profiles	El directorio completo de empleados debería requerir autenticación y, preferiblemente, un rol de gestión de personas.

Ejemplo de corrección mínima (Flask/Python)

```
@app.route('/profile')
@login_required
def profile():
    uid = request.args.get('uid', type=int)
    session_uid = session.get('uid')

    # Verificar que el usuario solicita su propio perfil
    # o que tiene un rol privilegiado
    if uid != session_uid and not current_user.is_admin:
        abort(403)  ← devuelve 403 Forbidden

    return render_profile(uid)
```

T+0:00 Inicio del laboratorio. Escaneo de red con `nmap -sn 172.17.0.0/16` → host `172.17.0.2` activo.

T+0:02 Port scan completo (`-p- -sV`) → servicio HTTP en el puerto 5000 (Flask/Werkzeug).

T+0:05 Exploración manual de la aplicación. `/about` filtra endpoints y arquitectura. `/profiles` expone el directorio completo sin auth.

T+0:08 Login con credenciales demo `alice/alice1`. Identificación de uid=17 (Carol CISO Admin) como objetivo.

T+0:12 Manipulación del parámetro: `/profile?uid=17`. Campo restringido visible → flag obtenida.

T+0:17 `BURP{idor_header_scope}` enviada. Lab completado en **17m 59s**.